

Introducción a Programación Funcional con TypeScript

Guía del Profesor — Tema 03

Matías Gel

Ciclo lectivo 2026

Índice de Contenidos

1	Guía del Profesor — Tema 03	2
1.1	Introducción a Programación Funcional con TypeScript	2
1.2	Resumen Ejecutivo	2
1.3	Índice de Artefactos	3
1.4	Plan de Clase Detallado	4
1.4.1	BLOQUE A0 — Historia: del Entscheidungsproblem al paradigma funcional	4
1.4.2	BLOQUE A — ¿Qué es el paradigma funcional?	5
1.4.3	BLOQUE B — Los tres pilares del paradigma funcional	6
1.4.4	BLOQUE C — TypeScript en modo funcional	8
1.4.5	BLOQUE D — Clojure: el funcional puro	10
1.4.6	BLOQUE E — Integración y cierre	12
1.5	Preguntas Frecuentes y Respuestas Anticipadas	13
1.5.1	Durante A0 (Historia)	13
1.5.2	Durante B (Tres pilares)	13
1.5.3	Durante C (TS funcional)	14
1.5.4	Durante D (Clojure)	14
1.6	Fragmentos Clave de los Libros	14
1.6.1	Gabbrielli & Martini — Capítulo 11 (fuente principal)	14
1.7	Gestión del Tiempo	15
1.8	Configuración del Aula	17
1.9	Notas Pedagógicas	17
1.9.1	Sobre el ritmo	17
1.9.2	Sobre los ejemplos con predicción	17
1.9.3	Sobre Clojure como “idioma extranjero”	17
1.9.4	Sobre la transparencia referencial	18

1 Guía del Profesor — Tema 03

1.1 Introducción a Programación Funcional con TypeScript

□ **Elaborada por:** Dr. Roberto (class-writer) **Materia:** Paradigmas y Lenguajes de Programación 2026 — UNTDF / IDEI — IF020 **Año:** 4°
Licenciatura en Sistemas **Semana:** 2 — Clase 1 **Duración:** 120 minutos

1.2 Resumen Ejecutivo

Esta clase introduce el **paradigma funcional** desde sus raíces matemáticas (λ -cálculo de Church, 1936) hasta su expresión práctica en TypeScript y Clojure. El hilo conductor es el *contraste*: el estudiante ya conoce el imperativo — la clase construye el funcional mostrando qué se gana al renunciar al estado mutable.

Mensaje central de la clase: > “El paradigma imperativo modela la computación como una Máquina de Turing — transformar el estado paso a paso. El funcional la modela como el λ -cálculo de Church — reducir una expresión a su valor. Aprender ambos no es aprender dos herramientas: es entender las dos formas fundamentales de formalizar la computación.”

Estructura en un vistazo:

Bloque	Tiempo	Qué logra
A0 — Historia	20 min	Ancla el paradigma en su origen matemático e histórico; activa conocimiento previo de Turing
A — Contraste paradigmas	10 min	Contraste directo imperativo vs funcional con código real
B — Tres pilares	20 min	Los tres conceptos fundamentales con ejemplos TS + Clojure
C — TypeScript funcional	25 min	map/filter/reduce en vivo; los alumnos predicen resultados

Bloque	Tiempo	Qué logra
D — Clojure	20 min	Lectura guiada del funcional puro; comparativa TS↔Clojure
E — Cierre	15 min	Debate socrático; pregunta de alto orden; anticipo próxima clase
Buffer	10 min	Preguntas, ajustes de tiempo

Artefactos de la clase: - filminas.md — [F-01] a [F-35] — 35 filminas schema-compliant - minuta.md — guión per-filmina, 35 secciones, 88 momentos ► - guia-estudio.md — guía autocontenida para el alumno (3-4 hs de estudio)

1.3 Índice de Artefactos

Archivo	Descripción	Uso
filminas.md	Plan de 35 filminas (sin imagen generada aún)	Proyectar durante la clase (exportar a Google Slides via <code>slides_pipeline.py</code>)
minuta.md	Guión per-filmina: momentos exactos, texto en voz alta, preguntas con respuestas guía	Abrir en pantalla secundaria o imprimir
guia-estudio.md	Guía del alumno: teoría, ejemplos, autoevaluación	Subir a Moodle antes de la clase
diseno.md	Diseño aprobado: OA, bloques, decisiones pedagógicas	Referencia de diseño; no usar en clase
topic.yaml	Estado del tema: <code>status: "class"</code>	Estado de workflow; no usar en clase

1.4 Plan de Clase Detallado

1.4.1 BLOQUE A0 — Historia: del Entscheidungsproblem al paradigma funcional

Tiempo: 20 minutos | **Filminas:** [F-01] a [F-08]

Objetivo del bloque: Situar el paradigma funcional en su contexto matemático-histórico. Activar conocimiento previo sobre Turing. Mostrar que el funcional resuelve problemas reales de la industria moderna.

[F-01] Portada (1 min) - Presentar el tema. Mencionar que esta clase tiene “dos capas”: historia y código. Preguntar: “*¿Alguien sabe de dónde viene el paradigma funcional —antes de que existieran las computadoras?*” - No develar la respuesta. Dejar la pregunta flotando.

[F-02] El Entscheidungsproblem (3 min) - Mostrar la pregunta de Hilbert (1928): > “*¿Existe un procedimiento mecánico que, dado cualquier enunciado matemático, determine de forma finita si es verdadero o falso?*” - Contextualizar: Hilbert quería matematizar toda la matemática. Dos personas respondieron en 1936, desde Cambridge y Princeton.

[F-03] Church vs Turing (4 min) - Mostrar la tabla comparativa Church/Turing. Insistir en el punto central: - Church: λ -cálculo $\rightarrow$ sustitución $\rightarrow$ paradigma **funcional** - Turing: Máquina de Turing $\rightarrow$ estados $\rightarrow$ paradigma **imperativo** - Ambos **prueban lo mismo** (no existe el algoritmo para el Entscheidungsproblem) pero con modelos opuestos. - **Pregunta guía:** “*¿Cuál de los dos es ‘mejor’?*” — Respuesta esperada: ninguno. Son equivalentes. Llevar a la Tesis Church-Turing.

[F-04] Tesis Church-Turing (2 min) - Todo lo computable por uno lo es por el otro. - Pero no son lo mismo filosóficamente: uno computa *modificando estados*, el otro *reescribiendo expresiones*.

[F-05] Recorrido de lenguajes funcionales (4 min) - Recorrer la tabla (1958 Lisp $\rightarrow$ 2012 Elixir). No leerla entera — resaltar: - Lisp (1958): primer lenguaje funcional, sigue vivo en Emacs Lisp - Erlang (1986): telecomunicaciones — WhatsApp corre en Erlang hoy - Haskell (1990): comité académico — finanzas cuantitativas, criptografía - Clojure (2007): blockchain, Datomic — el que vemos en esta clase - **Mensaje clave:** cada lenguaje de la lista nació para resolver un problema concreto de su momento histórico.

[F-06] Tabla multiparadigma (3 min) - Java 8 (2014) incorporó lambdas. JavaScript

ES6 (2015) arrow functions + .map(). TypeScript: todo eso + tipos estáticos + readonly. - **Pregunta:** “¿Por qué en los 2000s y no antes?” — Los CPUs dejaron de crecer en frecuencia y pasaron a múltiples núcleos. Estado mutable compartido → condiciones de carrera. El funcional las elimina por diseño.

[F-07] Frase de anclaje (1 min) > “Si el paradigma imperativo es la Máquina de Turing hecha lenguaje, el funcional es el λ -cálculo de Church hecho lenguaje.” Darle tiempo al alumno para anotarla o fotografiarla.

[F-08] Pregunta socrática de transición (2 min) - “Bien. Sabemos de dónde viene el funcional. Ahora: ¿cuál es la diferencia concreta en modo de computar? Veamos el mismo problema con los dos paradigmas.” - Transicionar sin pausa al Bloque A.

1.4.2 BLOQUE A — ¿Qué es el paradigma funcional?

Tiempo: 10 minutos | **Filminas:** [F-09] a [F-12]

Objetivo del bloque: Mostrar el contraste directo imperativo vs funcional con el ejemplo concreto de [1,2,3,4,5,6]. Introducir el modelo de β -reducción como mecanismo.

[F-09] Frase de anclaje (1 min) - Mostrar frase: > “Imperativo: computar = modificar el estado. Funcional: computar = reducir una expresión.” - Anunciar que lo van a ver en código ahora mismo.

[F-10] Código imperativo (3 min) - Mostrar:

```
const numeros = [1, 2, 3, 4, 5, 6];
let suma = 0;
for (let i = 0; i < numeros.length; i++) {
  if (numeros[i] % 2 === 0) {
    suma += numeros[i] * numeros[i];
  }
}
// suma = 56
```

- Señalar: let suma (estado mutable), i++ (modificar índice), suma += (modificar acumulador).
- Preguntar: “¿Cuántas variables mutamos durante la ejecución?” → suma + i → 2 variables en movimiento constante.

[F-11] Código funcional (4 min) - Mostrar:

```
const resultado = [1, 2, 3, 4, 5, 6]
  .filter(n => n % 2 === 0)    // [2, 4, 6]
  .map(n => n * n)              // [4, 16, 36]
  .reduce((acc, n) => acc + n, 0); // 56
```

- Preguntar *antes* de mostrar el resultado: “¿Qué devuelve `.filter(n => n % 2 === 0)`?” → esperar [2, 4, 6].
- Después: “¿Cuántas variables mutan?” → Ninguna. resultado es una sola asignación `const`.
- Señalar: mismo resultado (56), sin estado intermedio modificado.

[F-12] β -reducción (2 min) - Mostrar la β -reducción: $(\lambda n. n \% 2 === 0) \ 4 \rightarrow_{\beta} 4 \% 2 === 0 \rightarrow \text{true}$ $(\lambda n. n * n) \ 4 \rightarrow_{\beta} 4 * 4 \rightarrow 16$ - Conceptualizar: cada aplicación de función con `.map()` o `.filter()` es una β -reducción del λ -cálculo, en código moderno.

1.4.3 BLOQUE B — Los tres pilares del paradigma funcional

Tiempo: 20 minutos | **Filminas:** [F-13] a [F-18]

Objetivo del bloque: Que el alumno pueda enunciar y diferenciar los tres pilares con ejemplos en TS y Clojure. Este bloque tiene la mayor densidad conceptual — mantener el ritmo pausado.

[F-13] Los tres pilares (2 min) - Mostrar los tres términos. No definirlos aún — pedir a los alumnos que intenten una definición antes de mostrar la filmina siguiente. - “¿Qué imaginás que significa ‘función pura’?”

[F-14] Funciones puras — TS (4 min) - Mostrar el ejemplo `□/□` de incrementar vs sumar:

```
// □ Impura
let contador = 0;
const incrementar = () => { contador++; return contador; };

// □ Pura
const sumar = (a: number, b: number): number => a + b;
```

- Destacar: `sumar(3, 4)` hoy y dentro de un año devuelve 7. Siempre.
- Citar Gabbrielli-Martini: *“One may reason on those components in isolation, with the guarantee that they will always behave in the same manner, since no side-effect is around.”*

[F-15] Funciones puras — Clojure (2 min) - Mostrar contraparte:

```
;; □ Pura
(defn sumar [a b] (+ a b))
```

- Señalar: en Clojure *no existe* la posibilidad de hacer la versión impura de incrementar sin usar un mecanismo especial de estado (`atom`, `ref`) — la pureza es el default.

[F-16] Inmutabilidad (4 min) - Mostrar:

```
// □ Mutación oculta
let numeros = [1, 2, 3];
numeros.push(4); // numeros ahora es [1,2,3,4]

// □ Inmutable
const numeros = [1, 2, 3] as const;
const masNumeros = [...numeros, 4];
```

- Mostrar en Clojure: `(def numeros [1 2 3])` / `(def mas-numeros (conj numeros 4))` — `numeros` permanece `[1 2 3]`.
- Preguntar: *“¿Por qué usamos `const` y no `let`?”* — No es técnico: es disciplina de paradigma. `let` invita al estado.

[F-17] Transparencia referencial (5 min) - Este es el pilar más abstracto — darle tiempo. - Mostrar:

```
// □ Transparente
const doble = (n: number): number => n * 2;
doble(5) + doble(5) // → 10 + 10 → 20
10 + 10 // → 20 (equivalente — podemos sustituir)

// □ No transparente
let x = 0;
const siguiente = (): number => ++x;
siguiente() + siguiente() // → 1 + 2 → 3
1 + 1 // → 2 ≠ 3 (NO equivalente)
```

- Citar: *“This property, which is immediately falsified when there are side effects, is taken by many authors as the criterion for a pure functional language.”* — Gabbrielli-Martini, cap. 11.

[F-18] Resumen tres pilares (3 min) - Mostrar tabla resumen. Dar 30 segundos de silencio para que anoten. - Preguntar: *“¿Cuál de los tres les resulta más difícil de ver en código real?”* — Generalmente la transparencia referencial. Confirmar que es normal — es el más abstracto.

1.4.4 BLOQUE C — TypeScript en modo funcional

Tiempo: 25 minutos | **Filminas:** [F-19] a [F-26]

Objetivo del bloque: Los alumnos aprenden map, filter, reduce *prediciendo* el resultado antes de verlo. Construir la lógica de la composición funcional.

[F-19] map — problema imperativo (3 min)

```
const numeros = [1, 2, 3, 4, 5];
const dobles: number[] = [];
for (let i = 0; i < numeros.length; i++) {
  dobles.push(numeros[i] * 2);
}
```

- Señalar: estado mutable dobles, índice mutable i. ¿Qué operamos conceptualmente? “Transformar cada elemento”. Hay una operación esencial y mucho ruido.

[F-20] map — solución funcional (4 min)

```
const numeros = [1, 2, 3, 4, 5];
const dobles = numeros.map(n => n * 2);
// [2, 4, 6, 8, 10]
```

- Antes de mostrar el resultado: *“¿Qué devuelve [1,2,3,4,5].map(n => n*2)?”* — Esperar respuesta.
- Tipo: `Array<number>.map(n => n*2) : Array<number>` — transforma elementos, conserva la cantidad.

[F-21] filter (4 min)


```
const pares = [1, 2, 3, 4, 5, 6].filter(n => n % 2 === 0);
// [2, 4, 6]
```

- Antes de mostrar: “¿Qué devuelve `.filter(n => n % 2 === 0)` sobre `[1,2,3,4,5,6]`?”
- Tipo: conserva el tipo de los elementos, puede reducir la cantidad.
- Combinación: “Y si aplicamos `.map(n => n*n)` sobre `[2,4,6]`? ¿Cuánto da?” → `[4, 16, 36]`.

[F-22] reduce — problema (3 min)

```
const nums = [4, 16, 36];
let suma = 0;
for (let i = 0; i < nums.length; i++) {
  suma += nums[i];
}
// suma = 56
```

- Hay una operación esencial (acumular con `+`) y mucho ruido (índice, inicialización, loop).

[F-23] reduce — solución (4 min)

```
const suma = [4, 16, 36].reduce((acc, n) => acc + n, 0);
// 56
```

- Mostrar la traza:
 - paso 1: `(0, 4) → 4`
 - paso 2: `(4, 16) → 20`
 - paso 3: `(20, 36) → 56`
- Preguntar: “Si el valor inicial es 1 en lugar de 0, ¿cuál es el resultado?” → 57.
- Citar Gabbrielli: “`fold(fn x,y=>x+y, 0, list_of_int)` returns the sum of all the elements.”

[F-24] Chain filter → map → reduce (4 min)

```
const resultado = [1, 2, 3, 4, 5, 6]
  .filter(n => n % 2 === 0)
  .map(n => n * n)
  .reduce((acc, n) => acc + n, 0);
// 56
```

- Antes de mostrar: “Este código combina los tres. Sin ejecutarlo, ¿qué devuelve?”
- Marcar que esto es **composición de funciones** — la salida de cada operación es la entrada de la siguiente.

[F-25] pipe — composición explícita (3 min)

```
const pipe = (...fns: Array<(x: any) => any>) =>
  (x: any) => fns.reduce((acc, fn) => fn(acc), x);

const procesarNumeros = pipe(
  (nums: number[]) => nums.filter(n => n % 2 === 0),
  (nums: number[]) => nums.map(n => n * n),
  (nums: number[]) => nums.reduce((acc, n) => acc + n, 0)
);
procesarNumeros([1, 2, 3, 4, 5, 6]); // → 56
```

- Señalar: pipe se implementa con reduce — el meta-ejemplo: todo es función aplicada sobre función.

[F-26] const vs let — ¿por qué importa? (3 min) - Mostrar:

```
// □ Con let: la mente sigue el estado
let resultado = 0;
resultado = calcularParcial(datos);
resultado = ajustar(resultado);

// □ Con const: cada nombre es un valor definitivo
const parcial = calcularParcial(datos);
const final   = ajustar(parcial);
```

- Preguntar: “¿Qué diferencia hay en cómo lees el código?” — Con const, cada línea es una definición, no una instrucción que modifica algo.

1.4.5 BLOQUE D — Clojure: el funcional puro**Tiempo:** 20 minutos | **Filminas:** [F-27] a [F-32]

Objetivo del bloque: Mostrar que los mismos conceptos de TS en modo funcional existen en Clojure con la diferencia de que en Clojure *no hay otra opción*. La comparativa TS↔Clojure cierra los conceptos.

[F-27] Contexto Clojure (3 min) - Clojure = dialecto Lisp → JVM → inmutabilidad nativa → creado 2007 por Rich Hickey. - Nichos: concurrencia, blockchain (Datomic), finanzas. - “¿Por qué Lisp tiene paréntesis en todo?” — porque en Lisp el código es una lista. Las listas se procesan con funciones. El código y los datos comparten la misma sintaxis.

[F-28] Sintaxis de Clojure (4 min) - Leer despacio:

```
;; def = const en TS
(def numeros [1 2 3 4 5 6])

;; defn = función nombrada
(defn cuadrado [n] (* n n))

;; Función anónima
(fn [n] (* n n))    ; forma larga
#(* % %)           ; forma corta
```

- Preguntar: “¿Qué hace (* n n)?” → multiplica n por sí mismo.

[F-29] map/filter/reduce en Clojure (4 min)

```
(->> '(1 2 3 4 5 6)
      (filter even?)
      (map #(* % %))
      (reduce + 0))
; => 56
```

- Señalar ->>: pasa el resultado de cada expresión como último argumento de la siguiente.
- “¿Qué hace even??” → predicado booleano: true si el número es par. Equivalente a $n \Rightarrow n \% 2 === 0$.

[F-30] Comparativa TS ↔ Clojure (4 min) - Mostrar la tabla comparativa línea a línea:

Operación	TypeScript	Clojure
Valor const	const x = 5	(def x 5)
Función anónima	n => n * 2	#(* % 2)
map	arr.map(n => n*2)	(map #(* % 2) coll)
filter	arr.filter(n => n>2)	(filter #(> % 2) coll)
reduce	arr.reduce((a,n) => a+n, 0)	(reduce + 0 coll)

Operación	TypeScript	Clojure
Inmutabilidad	const (disciplina)	Nativa (imposible mutar)

- Preguntar: “¿Cuál es la diferencia más importante de la última fila?”

[F-31] Disciplina vs garantía (3 min) - TS: “*podés ser funcional si querés*” — disciplina del equipo - Clojure: “*no tenés otra opción*” — garantía del lenguaje - Preguntar: “*En un equipo grande, ¿qué preferirían?*” — Los que priorizan consistencia suelen preferir la garantía; los que priorizan flexibilidad prefieren el multiparadigma.

[F-32] Lisp en TS (2 min) - Citar Gabbrielli: “*In pure functional languages, there is neither a state nor a modifiable variable. The computation proceeds — at least in principle — by rewriting expressions.*” - Señal de transición: “*Vimos cómo surgió el paradigma, cómo se ve en TypeScript y cómo en Clojure puro. Antes de cerrar: una última pregunta de reflexión.*”

1.4.6 BLOQUE E — Integración y cierre

Tiempo: 15 minutos | **Filminas:** [F-33] a [F-35]

Objetivo del bloque: Consolidar con debate socrático. Anticipo de próxima clase. No hay nuevo contenido teórico — solo síntesis y reflexión.

[F-33] Línea de tiempo / resumen visual (3 min) - Mostrar la línea de tiempo completa de la clase: 1928 Hilbert → 1936 Church-Turing → 1958 Lisp → 1990 Haskell → 2007 Clojure → 2014 Java 8 / 2015 ES6 → hoy TypeScript. - Darle tiempo al alumno para anotarla.

[F-34] Pregunta de debate (8 min) - Mostrar la pregunta: > “*Si TypeScript puede ser funcional, ¿para qué aprender Clojure o Haskell? ¿Cuándo tiene sentido elegir un lenguaje puramente funcional sobre uno multiparadigma?*” - Dar 2 minutos para que los alumnos piensen en silencio → 5 minutos de debate. - **Guía de moderación:** - Si nadie habla: “*Piensen en un sistema bancario con concurrencia masiva. ¿Preferirían la disciplina o la garantía?*” - Si el debate se va al imperativo: redirigir — “*Estamos comparando dos tipos de funcional, no funcional vs imperativo.*” - Puntos a sacar si no aparecen: (1) escala del equipo → más personas, más valor la garantía; (2) dominio → blockchain, finanzas cuantitativas, telecomunicaciones prefieren pureza; (3) deuda técnica → en TS funcional mal aplicado es peor que TS imperativo limpio.

[F-35] Cierre y próxima clase (4 min) - Resumir los 3 puntos fundamentales de la clase: 1. El funcional es tan viejo como el imperativo — ambos vienen de 1936 2. Los tres pilares: funciones puras, inmutabilidad, transparencia referencial 3. map, filter, reduce son la forma concreta de programar sin estado - Anunciar próxima clase: **Profundización en funcional — Currying, closures, mónadas.** - Recordar: *“La guía de estudio ya está disponible en Moodle — 3-4 horas para consolidar.”*

1.5 Preguntas Frecuentes y Respuestas Anticipadas

1.5.1 Durante A0 (Historia)

P: ¿En qué consiste exactamente el λ -cálculo? R: *“Es un sistema formal con tres reglas: variables, abstracción ($\lambda x. expr$) y aplicación ($f(arg)$). Con esas tres reglas se puede expresar cualquier función computable. No es un lenguaje de programación — es un modelo matemático que inspira los lenguajes funcionales. Profundizamos en el β -cálculo en la clase de fundamentos teóricos.”*

P: ¿Haskell es mejor que TypeScript? R: *“Son herramientas para problemas distintos. Haskell garantiza pureza funcional, inferencia de tipos avanzada y evaluación lazy. TS tiene un ecosistema masivo, interoperabilidad con JavaScript y curva de entrada más baja. La pregunta correcta es: ¿para qué problema?”*

1.5.2 Durante B (Tres pilares)

P: ¿const en TypeScript garantiza inmutabilidad? R: *“Parcialmente. const impide la reasignación de la referencia, pero no impide mutar el contenido de un array u objeto. Para inmutabilidad profunda se necesita Object.freeze() o as const (para literales). Los tipos Readonly<T> y ReadonlyArray<T> la expresan en el sistema de tipos. Clojure va más lejos — no hay forma de mutar una colección sin usar primitivos especiales (atom, ref).”*

P: ¿Las funciones con I/O pueden ser puras? R: *“En sentido estricto, no — la I/O es un efecto colateral. Los lenguajes funcionales puros como Haskell modelan la I/O explícitamente con mónadas (IO monad), que actúan como ‘contenedores de efectos’ y los aíslan del código puro. En TypeScript práctico, la convención es mantener la I/O en el borde del sistema y tener todo el resto como puro. No es perfectamente puro, pero es mucho mejor que mezclar I/O en todas partes.”*

1.5.3 Durante C (TS funcional)

P: ¿reduce reemplaza a map y filter? R: “Técnicamente sí — se pueden implementar con reduce. Pero semánticamente no: usar map comunica ‘estoy transformando elemento a elemento’, usar filter comunica ‘estoy seleccionando’. reduce es el martillo si sólo tenés un martillo. Usar las operaciones más específicas hace el código más legible.”

P: ¿Por qué reduce((acc, n) => acc + n, 0) y no simplemente reduce((acc, n) => acc + n)? R: “El segundo argumento de reduce es el valor inicial del acumulador. Sin él, reduce usa el primer elemento del array como valor inicial — lo que puede causar errores en arrays vacíos. Con 0 el comportamiento es siempre definido.”

1.5.4 Durante D (Clojure)

P: ¿Clojure es lento por correr en la JVM? R: “El startup de la JVM es lento — lo que hace que Clojure sea percibido como lento en scripts cortos. Para servicios de larga duración (servidores, sistemas de procesamiento), el JIT de la JVM lo hace comparable a Java. ClojureScript (que compila a JavaScript) resuelve el problema en el navegador y Node. Rich Hickey diseñó Clojure para producción — Datomic, el sistema de base de datos de Cognitect, está escrito en Clojure.”

1.6 Fragmentos Clave de los Libros

1.6.1 Gabbrielli & Martini — Capítulo 11 (fuente principal)

Modelo de computación funcional puro: “In this chapter, we discuss the functional programming paradigm, where computation proceeds by term rewriting and not through modification of the state. Languages of this paradigm, at least in their ‘pure’ form, do not use the concept of memory.”

Contraste con el imperativo: “Conventional languages base their computational model on the transformation of the state. The heart of this model is the concept of modifiable variable.”

Transparencia referencial (definición estricta): “This property, which is immediately falsified when there are side effects, is taken by many authors as the criterion for a pure functional language: a language is purely functional if it satisfies this condition.”

Lenguajes puros: “In pure functional languages, there is neither a state nor a modifiable variable. The computation proceeds — at least in principle — by rewriting expressions.”

Funciones de orden superior y modularidad: “Programming in a functional style revolves around immutable data, manipulated by a (large) set of (small) functions. Using higher-order functions one may define general programming schemata as functions. The essential point is that the extensive use of program schemata increases the modularity of code.”

Aislamiento y testeo: “One may reason on those components in isolation, with the guarantee that they will always behave in the same manner, since no side-effect is around.”

Impacto del funcional en otros paradigmas: “Functional languages had a tremendous impact on the design of programming languages of any paradigm. Many concepts and experimental features in functional programming have later migrated to other paradigms. Among these concepts, type systems, generics, polymorphism, type inference...”

Origen histórico: “This is a paradigm as old as the imperative one. Since the 1930s, beside the Turing Machine, there has existed the λ -calculus, an abstract model for computable functions. Lisp was the first programming language explicitly based on this model.”

fold/reduce: “The function `fold(f, init, list_of_data)` is a (usually predefined) function that accumulates the elements of `list_of_data`, returning the accumulated result; `init` is the initial value used to start the fold.”

Estilos multiparadigma: “It is clear that one may use a functional programming style also using programming languages that allow for other programming paradigms. Once a language provides higher-order functions, it becomes easy to write large programs which avoid state-based computations.”

1.7 Gestión del Tiempo

Bloque	Presupuesto	Señal de alerta
A0 — Historia	20 min	Si llegas a [F-05] en más de 12 min, acelerará la tabla de lenguajes
A — Contraste	10 min	No extender — si hay preguntas sobre β -reducción, diferir a “fundamentos teóricos”
B — Tres pilares	20 min	Si [F-17] (transparencia referencial) genera confusión, dar 2 preguntas concretas en lugar de continuar la explicación
C — TS funcional	25 min	La predicción de resultados [F-20] y [F-21] no debería llevar más de 3 min. Si nadie responde, mover a explicación directa.
D — Clojure	20 min	Si el tiempo está ajustado, [F-31] y [F-32] se pueden fusionar (~3 min)
E — Cierre	15 min	Si el debate [F-34] se extiende, cortarlo a los 5 min con una síntesis a mano
Buffer	10 min	No lo sacrifiques para cubrir contenido — está para preguntas

Señal de que vas bien: Al llegar a [F-20] deberías estar entre los minutos 32 y 35 de clase.

Señal de que vas tarde: Si a los 45 min todavía estás en [F-17], comprímí los ejemplos Clojure de B y pasá al Bloque C directamente.

1.8 Configuración del Aula

Hardware mínimo: - Proyector o pantalla grande visible desde todos los ángulos - Si posible: segunda pantalla para la minuta (o imprimir minuta.md)

Configuración de código recomendada: - Editor abierto en una terminal con ts-node o REPL de TypeScript online (TypeScript Playground: typescriptrlang.org/play)
- Para Clojure live: tryclojure.org o REPL local si está configurado

Antes de entrar al aula: - Verificar que filminas.md tiene slides exportadas o preparar el Markdown para proyectar - Tener la minuta.md accesible (segunda pantalla o impresa) - Abrir el TypeScript Playground en el navegador para los ejemplos de live coding

1.9 Notas Pedagógicas

1.9.1 Sobre el ritmo

Esta clase tiene un bloque A0 de historia de 20 minutos antes de una línea de código. Es un bloque que puede percibirse como “larga introducción” por los alumnos. Para mantener la atención: - Mantener la pregunta “¿por qué el funcional entró en los lenguajes imperativos?” flotando desde [F-01] hasta [F-06] — cuando la respuesta llega (~minuto 18), genera la sensación de “arco narrativo”. - Usar la tabla de lenguajes como *contexto de industria*, no como lista a memorizar. Señalar dos o tres nombres reconocibles (WhatsApp → Erlang, JavaScript/TypeScript → ES6).

1.9.2 Sobre los ejemplos con predicción

El patrón de predicción (“¿qué devuelve este código antes de ejecutarlo?”) en el Bloque C tiene evidencia pedagógica de eficacia (Mayer 2023: principio de generación activa). Resistir la tentación de dar la respuesta antes de que los estudiantes respondan — incluso 10-15 segundos de silencio son valiosos.

1.9.3 Sobre Clojure como “idioma extranjero”

La sintaxis de Clojure puede generar rechazo inicial por la notación polaca (prefix) y los paréntesis. Estrategia recomendada: mostrar primero el *resultado* y recién después el código. “Si les digo que este código Clojure calcula 56, ¿pueden leer qué hace?” → luego mostrar el código.

1.9.4 Sobre la transparencia referencial

Es el concepto más abstracto de los tres pilares. Señal de que el grupo lo entendió: pueden responder *sin hesitar* por qué `siguiente()` + `siguiente()` no es equivalente a `1 + 1`. Si hay confusión persistente, usar el ejemplo bancario: “*Si `saldo()` devuelve el saldo de una cuenta, ¿es referencialmente transparente?*” → No, porque el saldo puede cambiar entre dos llamadas.

Documento generado por Dr. Roberto (class-writer) — EDU Academic Course Production Suite v1.0.0 Tema 03 — Paradigmas y Lenguajes de Programación 2026 — UNTDF/IDEI — IF020